KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

C PROJECT REPORT ON

Matrix Tools and Operations

SUBMITTED TO

The Technical Department

KG College of Arts and Science

SUBMITTED BY

Student Name	Roll No.	Course & Batch
Jayadhanushika	25BIT116	I-B.SC-IT-'B' (2025-2028)
Jeeva	25BIT117	I-B.SC-IT-'B' (2025-2028)
Jenileya	25BIT118	I-B.SC-IT-'B' (2025-2028)
Kabish	25BIT120	I-B.SC-IT-'B' (2025-2028)

Academic Year: **2025 – 2026**

Semester: **I**

DECLARATION

We hereby declare that the project entitled "**HOTEL & RESTAURANT BILLING SYSTEM"** submitted in partial fulfilment of the requirements for the course [**Course Name]** is a **record of our original work** carried out under the guidance of **[Trainer Name]**, Technical Trainer, Technical Department, KGiSL MicroCollege.

We further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

We have acknowledged all sources of information used in this project wherever applicable.

Student Name	Roll No.	Course & Batch	Signature
Jayadhanushika	25BIT116	I-B.SC-IT-'B' (2025-2028)	
Jeeva	25BIT117	I-B.SC-IT-'B' (2025-2028)	
Jenileya	25BIT118	I-B.SC-IT-'B' (2025-2028)	
Kabish	25BIT120	I-B.SC-IT-'B' (2025-2028)	

Place: COIMBATORE

Date: _____

CERTIFICATE

This is to certify that the project entitled **"Matrix Tools and Operations"** is a **bonafide work** carried out by the following students of **[Class/Batch]**, [**Department Name]**, **KG College of Arts & Science**, in partial fulfilment of the requirements for the course **[Course Name]**.

This project has been completed under my supervision and guidance during the academic period **[Month, Year]**.

It is further certified that this project is the **original work** of the students and has not been previously submitted for the award of any degree, diploma, or certificate at this or any other institution.

Project Team:

S. No.	Name	Roll Number
1.		
2.		
3.		
4.		

Guide:
(Technical Trainer, KGM)

Head of the Department
(Technical Head, KGM)

_____ _____

Place:

Date:

ABSTRACT

The project **“Matrix Tools and Operations”** is developed using the C programming language to perform various mathematical operations on matrices. The main objective of this project is to simplify matrix computations such as addition, subtraction, multiplication, and transpose using a structured and user-friendly approach.

The system allows users to input matrices of different sizes and perform operations through a menu-driven interface. It ensures accuracy in calculations and provides results instantly. The program demonstrates the use of arrays, functions, loops, and conditional statements in C language

This project helps in understanding mathematical problem-solving using programming and improves logical thinking. It is useful for students and beginners to learn matrix operations efficiently through coding.

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

1.INTRODUCTION

1.1 Objective

- To develop a Matrix Operations system using C programming
- To perform operations like addition, subtraction, multiplication, and transpose
- To demonstrate structured programming concepts using arrays and functions

1.2 Overview

Matrix operations are widely used in mathematics, engineering, and computer science. This system allows users to:

- Enter matrix elements
- Perform different matrix operations
- Display results in matrix format

1.3 Features

- Menu-driven program
- Supports multiple matrix operations
- Easy-to-understand output
- Efficient calculations Menu stored in arrays for easy access.

2. PROGRAM ANALYSIS

2.1 Problem Definition

Manual matrix calculations are time-consuming and error-prone. A system is required to:

- Accept matrix inputs
- Perform operations automatically
- Display accurate results

2.2 Project Overview

Aspect	Description
Language Used	C
Compiler	GCC / Dev-C++ / Turbo C++
Concepts Used	Arrays, Functions, loops
I/O Type	Console-based

2.3 Scopes and Limitations

Scope:

- Useful for students learning matrix operations
- Can be extended to advanced operation

Limitations:

- Works only in console mode.
- No graphical interface.
- Limited to basic operations

1. SYSTEM DESIGN

3.1 Modules

The Matrix Tools and Operations is divided into multiple functional modules, each handling a specific part of the program. These modules work together to make the system more efficient and user-friendly.

Module Name	Description
Input Module	Accepts matrix size and elements from the user
Addition Module	Add two matrices
Subtraction Module	Subtracts two matrices
Multiplication Mode	Prforms matrix multiplication
Transpose Module	Finds transpose of a matrix
Display Module	Displays matrix results

4.METHODOLOGY

The development of the project Matrix Tools and Operations follows a structured methodology that includes five main stages: **Problem Analysis, System Design, Program Development, Testing & Validation, and Results & Documentation.**

4.1. Problem Analysis

Matrix operations were analyzed and broken into smaller steps like input, processing, and output.

4.2. System Design

Matrices are stored using 2D arrays. Functions are used for each operation

4.3. Program Development

The program is written in C using Arrays for storing matrices Functions for Operations Loops for calculations

4.4. Testing and Validation

After coding, each module was tested individually to check correctness.

Tested with different matrix sizes Verified results manually Checked invalid inputs

4.5. Result and Documentation

The program successfully performs all matrix operations and displays correct outputs.

4.6. Algorithm

An **algorithm** is a **step-by-step logical procedure** used to solve a specific problem or perform a computation.

Step-by-step:

Start

Input matrix size

Enter matrix elements

Display menu

Choose operation

Perform selected operation

Display result

Stop

Purpose of the Algorithm

The main purpose of using an algorithm in this project is to:

- Define the **logical flow** of the billing operations.
- Ensure **accuracy** in calculations like subtotal, GST, and total amount.
- Provide a **structured path** for program development before actual coding.
- Make the system easy to understand, debug, and enhance in the future.

Algorithm in the Project

Step-by-Step Procedure:

Start the program

Declare variables

Declare matrices A, B, and Result

Declare variables for rows (r), columns (c), and choice

Input matrix size

Read number of rows and columns from the user

Input Matrix A

Use nested loops to read elements of matrix A

Input Matrix B

Use nested loops to read elements of matrix B

Display menu

Show options:

Addition

Subtraction

Multiplication

Transpose

Read user choice

Perform operation using switch case

If choice = 1 (Addition):

For each element

Result[i][j] = A[i][j] + B[i][j]

If choice = 2 (Subtraction):

Result[i][j] = A[i][j] - B[i][j]

If choice = 3 (Multiplication):

Use three nested loops

Result[i][j] = Σ (A[i][k] × B[k][j])

If choice = 4 (Transpose):

Transpose[j][i] = A[i][j]

Else:

Display "Invalid choice"

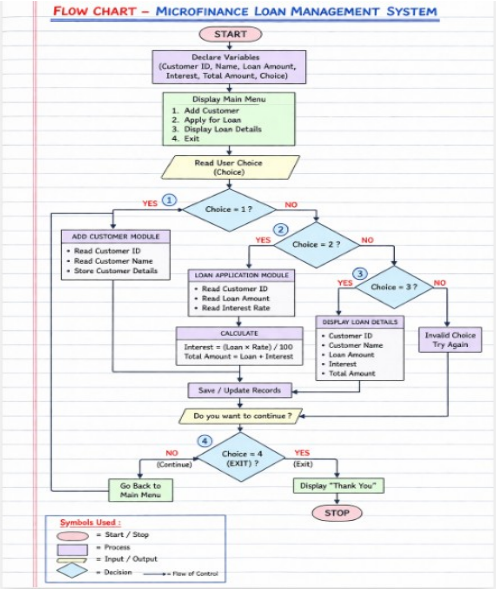
Display result matrix

4.7. Flowchart

The flowchart represents the logical flow of the *Matrix Tools and Opertions*. It visually outlines the sequence of operations — from displaying the menu to generating the customer bill — ensuring clarity in how data moves through each stage of the program.

Explanation of Flowchart:

1.Start The program begins execution.
2.Input Rows and Columns User enters: Number of rows Number of columns This defines the matrix
3.Input Matrix A User enters elements of first matrix using loops.
4.Input Matrix B User enters elements of second matrix.
5.Display Menu Program shows available operations: Addition, Subtraction, Multiplication, Transpose
6.Enter Choice User selects the operation using a number (1–4).
7.Decision Making (Switch Case)Based on user choice:
➤If Choice = 1 (Addition)Add corresponding elements Result[i][j] = A[i][j] + B[i][j]


FLOW CHART – MICROFINANCE LOAN MANAGEMENT SYSTEM
START
Declare Variables
(Customer ID, Name, Loan Amount, Interest, Total Amount, Choice)
Display Main Menu
1. Add Customer
2. Apply for Loan
3. Display Loan Details
4. Exit
Read User Choice
(Choice)
1
Choice = 1 ?
YES
NO
ADD CUSTOMER MODULE
• Read Customer ID
• Read Customer Name
• Store Customer Details
2
Choice = 2 ?
YES
NO
LOAN APPLICATION MODULE
• Read Customer ID
• Read Loan Amount
• Read Interest Rate
CALCULATE
Interest = (Loan × Rate) / 100
Total Amount = Loan + Interest
3
Choice = 3 ?
YES
NO
DISPLAY LOAN DETAILS
• Customer ID
• Customer Name
• Loan Amount
• Interest
• Total Amount
Invalid Choice
Try Again
Save / Update Records
Do you want to continue ?
4
Choice = 4
(EXIT) ?
NO
(Continue)
YES
(Exit)
Go Back to
Main Menu
Display "Thank You"
STOP
Symbols Used :
= Start / Stop
= Process
= Input / Output
= Decision
= Flow of Control

5.SOURCE CODE

```
  #include <stdio.h>

#define MAX 10

void inputMatrix(int matrix[MAX][MAX], int r, int c) {
   for(int i=0;i<r;i++) {
      for(int j=0;j<c;j++) {
         printf("Enter element [%d][%d]: ", i, j);
         scanf("%d", &matrix[i][j]);
      }
   }
}

void displayMatrix(int matrix[MAX][MAX], int r, int c) {
   for(int i=0;i<r;i++) {
      for(int j=0;j<c;j++) {
         printf("%d ", matrix[i][j]);
      }
      printf("\n");
   }
}

void addMatrix(int a[MAX][MAX], int b[MAX][MAX], int r, int c) {
   int result[MAX][MAX];
   for(int i=0;i<r;i++)
      for(int j=0;j<c;j++)
         result[i][j] = a[i][j] + b[i][j];

   printf("\nResult (Addition):\n");
   displayMatrix(result, r, c);
}
```

```
void subtractMatrix(int a[MAX][MAX], int b[MAX][MAX], int r, int c) {
   int result[MAX][MAX];
   for(int i=0;i<r;i++)
      for(int j=0;j<c;j++)
         result[i][j] = a[i][j] - b[i][j];

   printf("\nResult (Subtraction):\n");
   displayMatrix(result, r, c);
}

void multiplyMatrix(int a[MAX][MAX], int b[MAX][MAX], int r1, int c1, int c2) {
   int result[MAX][MAX];
   for(int i=0;i<r1;i++) {
      for(int j=0;j<c2;j++) {
         result[i][j] = 0;
         for(int k=0;k<c1;k++) {
            result[i][j] += a[i][k] * b[k][j];
         }
      }
   }

   printf("\nResult (Multiplication):\n");
   displayMatrix(result, r1, c2);
}

void transposeMatrix(int matrix[MAX][MAX], int r, int c) {
   int transpose[MAX][MAX];
   for(int i=0;i<r;i++)
      for(int j=0;j<c;j++)
         transpose[j][i] = matrix[i][j];

   printf("\nTranspose:\n");
   displayMatrix(transpose, c, r);
}

int main() {
   int a[MAX][MAX], b[MAX][MAX];
   int r, c, choice;
```

```
    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);

    printf("\nEnter Matrix A:\n");
    inputMatrix(a, r, c);

    printf("\nEnter Matrix B:\n");
    inputMatrix(b, r, c);

    printf("\n1. Addition\n2. Subtraction\n3. Multiplication\n4. Transpose\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch(choice) {
        case 1: addMatrix(a,b,r,c); break;
        case 2: subtractMatrix(a,b,r,c); break;
        case 3: multiplyMatrix(a,b,r,c,c); break;
        case 4: transposeMatrix(a,r,c); break;
        default: printf("Invalid choice");
    }

    return 0;
}
```

SOURCE CODE EXPLANATION:

1) Headers & Macros

#include <stdio.h>
#define MAX 10

Explanation:
stdio.h → Used for input and output functions like printf() and scanf().
#define MAX 10 → Defines the maximum size of the matrix (10×10).
This avoids using large memory and keeps the program controlled.

---

2) Function: Input Matrix

void inputMatrix(int matrix[MAX][MAX], int r, int c)

- Purpose:
  To read matrix elements from the user.
  Working:
  Uses nested loops:
  Outer loop → rows
  Inner loop → columns
  Each element is stored as
- scanf("%d", &matrix[i][j]);
- Logic:
  It fills the matrix row by row.

---

3) Function: Display Matrix

void displayMatrix(int matrix[MAX][MAX], int r, intc)
Purpose:
To print matrix in proper format.
Working:
Uses nested loops to print each element.
printf("%d ", matrix[i][j]);
Output Format:

1 2 3
4 5 6

---

4) Function: Addition of Matrices
void addMatrix(int a[MAX][MAX], int b[MAX][MAX], int r, int c)
Purpose:
To add two matrices.
Formula:
Result[i][j] = A[i][j] + B[i][j]
Working:
Loops through each element.
Adds corresponding elements of both matrices.
Stores result in a new matrix.

---

5) Function: Subtraction of Matrices

void subtractMatrix(int a[MAX][MAX], int b[MAX][MAX], int r, int c)
Purpose:

To subtract matrices

\

Formula:

Result[i][j] = A[i][j] – B[i][j]

Working:
Similar to addition.
Subtracts element by element.

---

6) Function: Multiplication of Matrices
void multiplyMatrix(int a[MAX][MAX], int b[MAX][MAX], int r1, int c1, int c2)
Purpose:
To multiply two matrices.
Condition:
Number of columns of A = Number of rows of B
Formula:
Result[i][j] = Σ (A[i][k] × B[k][j])

Working:
Uses three nested loops:
i → rows of A
j → columns of B
k → multiplication process

---

7) Function: Transpose of Matrix
void transposeMatrix(int matrix[MAX][MAX], int r, int c)
Purpose:
To convert rows into columns.
Formula:
Transpose[j][i] = Matrix[i][j]
Working:
Swaps row index with column index
1 2 1 3
3 4 → 2 4

---

8)Main Function
int main()
Variables:
int a[MAX][MAX], b[MAX][MAX];
int r, c, choice;
a, b → Matrices
r, c → Rows and columns
choice → User option

---

9) Input Section
scanf("%d %d", &r, &c);
User enters matrix size.
inputMatrix(a, r, c);
inputMatrix(b, r, c);
Calls function to input matrices

---

10) Menu Display

printf("1. Addition\n2. Subtraction\n3. Multiplication\n4. Transpose\n");

Displays list of operations.

Control Flow Summary

1. Start program
2. Input matrix size
3. Input matrix A and B
4. User selects operation
5. Perform operation
6. Display result
7. End the program

INPUT AND OUTPUT SCREENS

Output Screen:



DISCUSSION & CONCLUSION

7.1 Discussion

The project demonstrates:
Use of arrays and functions
Efficient matrix computation
Structured programming

7.2 Conclusion

The project demonstrates:
Use of arrays and functions
Efficient matrix computation
Structured programming

REFERENCES

- *The C Programming Language – Kernighan & Ritchi*
- *GeeksforGeeks – Matrix Operations in C*
- *TutorialsPoint – C Programming*
- Dev-C++ User Manual